Python — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Installation

02 Variables and Data Types

03 Operators and Input/Output

04 Conditions and Loops

05 Functions

06 Lists, Tuples, Sets, Dictionaries

07 Strings and String Methods

Intermediate (6 topics)

08 File Handling

09 Exception Handling

10 Modules, Packages and pip

11 Object-Oriented Programming

12 Iterators, Generators, Decorators

13 Standard Library Highlights

Advanced (6 topics)

14 Type Hints and Static Checking

15 Concurrency: Threads, Processes, asyncio

16 Working with APIs and Databases

17 Testing with pytest

18 Web Frameworks Overview

19 Data Analysis: NumPy and Pandas

Master (4 topics)

20	Packaging and Project Structure
21	Performance and Profiling
22	Design Patterns and Clean Code
23	Real Projects

Beginner Level

01. Introduction and Installation

Python is a readable, general-purpose language used for web, automation, data science and AI. Install Python 3 and run `python file.py`.

```
print('Hello, Python!')
```

02. Variables and Data Types

Dynamic typing: `int`, `float`, `str`, `bool`, `None`. Use `type()`; f-strings format output.

```
name = 'Priya'
age = 22
print(f'{name} is {age}')
```

03. Operators and Input/Output

Arithmetic (`//`, `%`, `**`), comparison, logical operators, `input()` and `print()`.

```
n = int(input('Number: '))
print(n ** 2)
```

04. Conditions and Loops

`if/elif/else`, `for`, `while`, `range`, `break`, `continue`, `enumerate`, `zip`.

```
for i, fruit in enumerate(['apple', 'mango'], start=1):
    print(i, fruit)
```

05. Functions

Define with `def`; default, keyword and `*args/**kwargs` parameters; lambdas; docstrings.

```
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}'
```

06. Lists, Tuples, Sets, Dictionaries

Lists are mutable sequences, tuples immutable, sets unique values, dicts key-value maps. Comprehensions create them concisely.

```
squares = {n: n * n for n in range(5)}
evens = [n for n in range(10) if n % 2 == 0]
```

07. Strings and String Methods

Slicing, `split`, `join`, `strip`, `replace`, `find`, formatting.

```
'a,b,c'.split(',')   # ['a', 'b', 'c']
'python'[::-1]       # 'nohtyp'
```

Intermediate Level

08. File Handling

Use `with open()` for safe file access; work with CSV and JSON modules.

```
with open('data.txt', 'w') as f:
    f.write('hello')
```

09. Exception Handling

`try/except/else/finally`, raise exceptions, custom exception classes.

```
try:
    x = int('abc')
except ValueError as e:
    print('Bad number', e)
```

10. Modules, Packages and pip

Import code from files and packages; manage dependencies with `pip` and virtual environments.

```
python -m venv .venv
source .venv/bin/activate
pip install requests
```

11. Object-Oriented Programming

Classes, `__init__`, inheritance, polymorphism, `@property`, dunder methods, dataclasses.

```
from dataclasses import dataclass
@dataclass
class Point:
    x: int
    y: int
```

12. Iterators, Generators, Decorators

Generators `yield` values lazily; decorators wrap functions to add behaviour.

```
def timer(fn):
    def wrapper(*a, **k):
        print('calling', fn.__name__)
        return fn(*a, **k)
    return wrapper
```

13. Standard Library Highlights

`datetime`, `pathlib`, `collections`, `itertools`, `re`, `json`, `logging`, `argparse`.

```
from collections import Counter
Counter('banana').most_common(1)
```

Advanced Level

14. Type Hints and Static Checking

Annotate with `list[int]`, `dict[str, int]`, `Optional`; check with mypy/pyright.

```
def total(nums: list[int]) -> int:
    return sum(nums)
```

15. Concurrency: Threads, Processes, asyncio

Threads suit I/O, processes suit CPU work, `asyncio` handles many I/O tasks with `async/await`.

```
import asyncio
async def main():
    await asyncio.gather(asyncio.sleep(1), asyncio.sleep(1))
asyncio.run(main())
```

16. Working with APIs and Databases

`requests` / `httpx` for HTTP; `sqlite3`, SQLAlchemy for databases.

```
import requests
r = requests.get('https://api.github.com')
print(r.status_code)
```

17. Testing with pytest

Write small test functions, fixtures and parametrised tests.

```
def test_add():
    assert add(2, 3) == 5
```

18. Web Frameworks Overview

Flask (micro), Django (batteries included), FastAPI (modern async APIs).

```
pip install fastapi uvicorn
```

19. Data Analysis: NumPy and Pandas

Vectorised arrays and DataFrames for cleaning and analysing data.

```
import pandas as pd
df = pd.read_csv('sales.csv')
df.groupby('city')['amount'].sum()
```

Master Level

20. Packaging and Project Structure

`pyproject.toml`, src layout, linting (ruff), formatting (black), pre-commit, publishing to PyPI.

```
ruff check . && pytest
```

21. Performance and Profiling

Profile with `cProfile`, use built-ins, avoid premature optimisation, NumPy, caching with `functools.lru_cache`.

```
from functools import lru_cache
@lru_cache
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

22. Design Patterns and Clean Code

PEP 8, SOLID, composition over inheritance, dependency injection, context managers.

```
class Timer:
    def __enter__(self): ...
    def __exit__(self, *exc): ...
```

23. Real Projects

CLI expense tracker, web scraper, REST API, data dashboard, automation scripts with scheduling.

Interview Question Bank

Q1. List vs tuple?
Lists are mutable; tuples are immutable and hashable (if contents are).

Q2. What is a decorator?
A function that takes a function and returns an enhanced one, applied with `@`.

Q3. `is` vs `==`?
`==` compares values; `is` compares object identity.

Q4. What are generators?
Functions with `yield` that produce values lazily, saving memory.

Q5. Explain the GIL.
CPython's global interpreter lock lets one thread run Python bytecode at a time; use processes or async for scaling.

Q6. Mutable default argument pitfall?
Defaults are evaluated once; `def f(x=[])` shares the list between calls - use `None`.

Q7. What is a context manager?
An object with `__enter__`/`__exit__` used by `with` for setup/cleanup.

Q8. `*args` and `**kwargs`?
Collect extra positional and keyword arguments.

Q9. List comprehension vs generator expression?
A list comprehension builds a list now; a generator expression yields items lazily.

Q10. What is a virtual environment?
An isolated Python environment with its own installed packages.

Capstone Project Ideas

1. CLI expense tracker
2. Web scraper with BeautifulSoup
3. Weather app using API
4. Automated email sender
5. Data analysis dashboard